

Konfigurasi Koneksi Basis Data PostgreSQL–Prisma ORM & Deployment Aplikasi ke Vercel

Draf Materi Subbab Implementasi Sistem

Draf langkah-langkah teknis untuk melengkapi subbab Implementasi Basis Data dan Implementasi Instalasi Program pada Bab 4 skripsi. Naskah ini masih perlu dibaca ulang, disesuaikan gaya bahasanya, dan dilengkapi tangkapan layar asli sebelum ditempel ke dokumen akhir.

1 Konfigurasi Koneksi Basis Data PostgreSQL ke Prisma ORM

Sistem SIAKAD PPIQ-368 menggunakan PostgreSQL sebagai basis data relasional dan Prisma ORM sebagai lapisan akses data. Koneksi antara keduanya tidak dituliskan langsung sebagai connection string pada berkas skema, melainkan dibaca dari satu variabel lingkungan (`DATABASE_URL`) yang dipakai konsisten baik oleh aplikasi saat berjalan maupun oleh perkakas baris perintah Prisma. Berikut langkah penyiapannya.

1. Menyiapkan basis data PostgreSQL. Tersedia dua pilihan: menggunakan layanan basis data terkelola (mis. Prisma Postgres melalui `console.prisma.io`, atau penyedia lain seperti Neon/Supabase), atau memasang PostgreSQL pada peladen/komputer sendiri. Untuk proyek ini disiapkan tiga basis data terpisah sesuai lingkungan: pengembangan lokal, pratinjau (preview), dan produksi — agar perubahan skema saat pengembangan tidak memengaruhi data yang sudah berjalan di produksi.
2. Mencatat connection string yang diberikan oleh layanan/peladen basis data, dengan format umum `postgresql://USER:PASSWORD@HOST:PORT/DATABASE?schema=public`.
3. Menyalin berkas `.env.example` pada direktori proyek menjadi `.env`, lalu mengisi variabel `DATABASE_URL` dengan connection string dari langkah sebelumnya.

Tangkapan layar disarankan: isi berkas `.env` dengan nilai kredensial disamarkan/dihitamkan sebelum discreenshot.

4. Mendeklarasikan datasource pada `prisma/schema.prisma` dengan provider `postgresql`. Koneksi aktualnya dibentuk lewat driver adapter `@prisma/adapter-pg`: pada kode aplikasi (`lib/prisma.ts`) adapter ini membaca `process.env.DATABASE_URL` untuk membuat instance Prisma Client, sedangkan pada operasi CLI (migrasi, seed, studio) nilai yang sama dibaca oleh `prisma.config.ts`. Berkas konfigurasi ini juga sengaja digagalkan secara eksplisit apabila `DATABASE_URL` kosong, supaya kesalahan konfigurasi terlihat jelas alih-alih diam-diam jatuh ke nilai baku yang salah.
5. Menjalankan pustaka dan Prisma Client sesuai skema:

```
pnpm install
pnpm exec prisma generate
```

6. Menerapkan skema ke basis data. Untuk lingkungan pengembangan, migrasi dibuat sekaligus diterapkan:

```
pnpm exec prisma migrate dev --name init
```

Untuk lingkungan yang riwayat migrasinya sudah ada (mis. staging/produksi), migrasi yang sudah tercatat cukup diterapkan tanpa membuat migrasi baru:

```
pnpm exec prisma migrate deploy
```

7. Mengisi data uji (opsional, untuk kebutuhan demo/pengujian):

```
pnpm exec prisma db seed
```

8. Memverifikasi koneksi dan melihat isi tabel melalui antarmuka visual Prisma Studio:

```
pnpm exec prisma studio
```

Perintah ini membuka jendela browser pada `localhost:5555` yang menampilkan seluruh tabel hasil migrasi beserta datanya.

Tangkapan layar disarankan: tampilan Prisma Studio dengan salah satu tabel terbuka (mis. tabel `Admission` atau `User`).

Catatan: berkas `.env` tidak pernah diikutsertakan ke repositori (sudah masuk `.gitignore`) karena memuat kredensial basis data. Nilainya hanya disalin manual di komputer pengembang, dan pada Vercel diisi lewat Environment Variables sebagaimana dijelaskan pada bagian berikut.

2 Deployment Aplikasi ke Vercel

Setelah basis data tersambung dan aplikasi berjalan normal di lingkungan lokal, tahap berikutnya adalah menerbitkan aplikasi ke internet melalui Vercel. Skrip build proyek ini (`package.json`) sudah dirancang untuk menerapkan migrasi basis data sebelum membangun aplikasi:

```
"build": "prisma migrate deploy && prisma generate && next build"
```

Artinya setiap kali Vercel menjalankan build, skema basis data produksi otomatis disinkronkan dengan migrasi terbaru sebelum aplikasi Next.js dibangun. Ada dua cara untuk melakukan

deployment: melalui antarmuka dashboard di peramban, atau melalui Vercel CLI. Keduanya menghasilkan hasil akhir yang sama.

2.1 Melalui Dashboard Vercel (Peramban)

1. Memastikan kode sumber sudah didorong (push) ke repositori GitHub proyek.
2. Membuka `vercel.com`, masuk (login) menggunakan akun yang tertaut ke GitHub, lalu memilih **Add New** → **Project**.

Tangkapan layar disarankan: halaman dashboard Vercel — tombol Add New Project.

3. Memilih **Import Git Repository** dan menunjuk repositori sistem ini. Vercel otomatis mengenali proyek sebagai aplikasi Next.js dan mengisi build command serta output directory secara baku.

Tangkapan layar disarankan: layar Import Project dengan repositori terpilih dan framework preset "Next.js".

4. Pada bagian **Environment Variables**, menambahkan variabel `DATABASE_URL` dan `NEXT_PUBLIC_APP_NAME`, masing-masing untuk ketiga environment yang disediakan Vercel (Production, Preview, Development) sesuai basis data tujuan tiap environment.

Tangkapan layar disarankan: form Environment Variables terisi (nilai `DATABASE_URL` disamarkan).

5. Menekan tombol **Deploy** dan menunggu proses build selesai.
6. Setelah build berhasil (status *Ready*), aplikasi dapat diakses melalui domain yang diberikan Vercel.

Tangkapan layar disarankan: halaman ringkasan deployment berstatus "Ready" beserta domainnya.

7. Sejak proyek tertaut ke repositori GitHub, setiap *push* berikutnya ke branch `main` otomatis memicu deployment produksi baru, sedangkan push ke branch lain menghasilkan deployment pratinjau dengan domain unik tersendiri — berguna untuk menguji perubahan sebelum digabungkan ke `main`.

2.2 Melalui Vercel CLI

1. Memasang Vercel CLI secara global:

```
pnpm add -g vercel
```

2. Masuk ke akun Vercel dari terminal:

```
vercel login
```

Perintah ini membuka alur autentikasi (mis. lewat tautan yang dikirim ke email atau login GitHub) yang dikonfirmasi langsung dari terminal.

3. Menautkan direktori proyek lokal ke proyek Vercel:

```
vercel link
```

Perintah ini membuat berkas `.vercel/project.json` yang menyimpan `projectId` dan `orgId`, sehingga perintah CLI berikutnya otomatis merujuk ke proyek yang benar tanpa perlu dipilih ulang.

4. Menambahkan variabel lingkungan yang dibutuhkan aplikasi untuk tiap environment:

```
vercel env add DATABASE_URL production
vercel env add DATABASE_URL preview
vercel env add DATABASE_URL development
vercel env add NEXT_PUBLIC_APP_NAME production
```

Variabel yang sudah ditambahkan dapat diperiksa kembali dengan `vercel env ls`.

5. Menjalankan deployment pratinjau lebih dulu untuk verifikasi sebelum ke produksi:

```
vercel
```

6. Setelah pratinjau dipastikan berjalan baik, menerbitkan ke produksi:

```
vercel --prod
```

Tangkapan layar disarankan: output terminal `vercel --prod` yang menampilkan status build dan URL produksi.

Catatan: pada sistem ini, kedua jalur di atas bermuara ke proyek Vercel yang sama (`pesantren-lms`) dan sudah tertaut ke repositori GitHub `mrramdn/SIAKAD-PPIQ-368` , sehingga alur yang berjalan sehari-hari adalah *continuous deployment*: cukup push ke branch `main` , Vercel menjalankan build (termasuk `prisma migrate deploy`) dan menerbitkan versi terbaru secara otomatis. Aplikasi produksi dapat diakses pada domain `pesantren-lms-seven.vercel.app` .

Variabel	Kegunaan	Diisi untuk
<code>DATABASE_URL</code>	Connection string PostgreSQL yang dibaca Prisma Client & Prisma CLI	Production, Preview, Development
<code>NEXT_PUBLIC_APP_NAME</code>	Nama aplikasi yang tampil pada antarmuka	Production, Preview, Development

Naskah ini adalah draf teknis, bukan teks akhir — tetap perlu diedit dengan kalimat sendiri dan dilengkapi tangkapan layar asli dari aplikasi/dashboard yang benar-benar dijalankan sebelum ditempel ke Bab 4.